madsql User Guide

`madsql` is a deterministic command-line tool for converting SQL between dialects, splitting multi-statement scripts into stable per-statement files, and inferring starter schema DDL or JSON from SQL workloads.

This guide is written for users who need to move SQL across database implementations, break large SQL assets into reviewable units, or recover useful schema structure from existing query collections.

Before You Begin

Install from the repository root:

```
python -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -e .
```

If you plan to use hybrid schema inference with `sql-metadata` and `simple-ddl-parser`, install the optional infer extras:

```
pip install -e .[infer]
```

Verify the runtime:

```
madsql --version
```

Inspect supported dialect names:

```
madsql dialects
```

If the console script is not on your shell `PATH`, run the module directly:

```
python3 -m madsql --help
```

Contents

Chapter 1: Why This Tool Exists

Migrating SQL across database implementations looks simple until the work stops being one query and becomes a repository, a vendor drop, or a multi-team application. SQL dialects diverge in all the places that matter to migration:

- Identifier quoting rules
- Pagination syntax such as `TOP` versus `LIMIT`
- Function names and argument order
- DDL capabilities and data type naming
- Schema and user management semantics
- Statement terminators and batch layout

Manual rewriting can work for a handful of queries, but it breaks down quickly when you need repeatability. Teams usually need more than a translator:

- They need the same input to produce the same output every time.
- They need batch processing across files, directories, and globs.
- They need clean output naming so diffs stay readable.
- They need artifacts that help them review failures without rerunning the job interactively.
- They sometimes need to infer schema structure from workloads because the source DDL is incomplete, unavailable, or no longer trustworthy.

`madsql` exists to solve that workflow problem.

It is strongest when you want deterministic automation around SQL migration, normalization, and inspection. It is not a GUI and it does not claim to replace database-level testing. It gives you stable, scriptable behavior at the file and statement level so you can move faster with less manual cleanup.

At a high level:

- Use `split-statements` when statement boundaries are the primary output.
- Use `convert` when translated SQL is the primary output.
- Use `infer-schema` when recovered DDL or schema JSON is the primary output.
- Use `--errors`, `--log`, and `--report` when the run itself needs to be inspectable and repeatable.

Chapter 2: Split Statements

`split-statements` is the split-only workflow. Use it when a large SQL file is hard to review, when you need one file per statement for selective replay, or when you want to prepare input for downstream inspection without changing the dialect.

When To Use It

Common cases:

- Breaking vendor SQL bundles into reviewable chunks
- Turning migration scripts into one-file-per-statement assets
- Isolating parse problems to a specific statement index

- Preparing SQL for code review or selective execution

Basic Usage

Split one file:

```
madsql split-statements --in ./input.sql --out ./split
```

Split a directory tree:

```
madsql split-statements --in ./sql --out ./split
```

Split from standard input:

```
cat input.sql | madsql split-statements --out ./split
```

If the input uses dialect-specific syntax, provide a source dialect hint:

```
madsql split-statements --source tsql --in ./queries.sql --out ./split
```

How Output Is Organized

`split-statements` always requires `--out` because it writes one file per statement.

Important output rules:

- Output files use deterministic names such as `0001_stmt.sql`, `0002_stmt.sql`, and so on.
- Directory inputs preserve their relative structure inside `--out`.
- Single-file inputs under the current working directory preserve their relative path under `--out`.
- Absolute inputs outside the current working directory are mapped under an `_external/...` prefix so the layout remains deterministic instead of flattening filenames.

Example layout:

```
./split/examples/input/mssql/example_queries/0001_stmt.sql
./split/examples/input/mssql/example_queries/0002_stmt.sql
./split/examples/input/mssql/example_queries/0003_stmt.sql
```

Rendering Behavior

By default, rendered SQL is compact. Use `--pretty` when you want multiline formatting:

```
madsql split-statements --source postgres --in ./queries.sql --out ./split --pretty
```

Use `--compact` if you want to be explicit about the default:

```
madsql split-statements --source postgres --in ./queries.sql --out ./split --compact
```

Parse Fallback

The command tries `SQLGlot` first. If `SQLGlot` cannot parse a split-only input, `madsql` can fall back to `sqlparse` for boundary detection on that input. This is useful when exact conversion is not required but stable statement boundaries still are.

The fallback matters for two reasons:

- You can still get per-statement files when full dialect parsing fails.
- Pretty rendering depends on `SQLGlot`; fallback splitting focuses on boundary detection, not higher-fidelity re-rendering.

Split While Also Inferring Schema

You can generate a schema side artifact during the split run:

```
madsql split-statements \
  --source postgres \
  --in ./sql \
  --out ./split \
  --infer-schema \
  --infer-schema-format json
```

Artifact tuning options for this workflow include:

- `--infer-schema-format ddl|json`
- `--infer-schema-engine sqlglot|hybrid`
- `--infer-schema-default-type TYPE`
- `--infer-schema-unqualified-columns first-table|skip`
- `--infer-schema-if-not-exists`
- `--infer-schema-create-schema`
- `--infer-schema-create-user`
- `--infer-schema-create-user-password PASSWORD`

`--infer-schema-engine hybrid` keeps SQLGlot as the primary inference engine and supplements the side artifact with `sql-metadata` and `simple-ddl-parser`. Hybrid mode requires `pip install -e .[infer]`.

Chapter 3: Convert

`convert` is the main migration workflow. Use it when your primary output is translated SQL in a target dialect and optional side artifacts are secondary.

What Convert Does

`convert` reads SQL from standard input, a file, a directory tree, or a glob pattern and renders translated SQL into the target dialect.

The minimum required flags are:

- `--source`
- `--target`

Fastest Example

Convert one T-SQL statement from standard input to PostgreSQL:

```
echo 'SELECT TOP 3 [name] FROM dbo.users;' | madsql convert --source tsql --target oracle
```

Output:

```
SELECT "name" FROM dbo.users FETCH FIRST 3 ROWS ONLY
```

Input Modes

Single file to standard output:

```
madsql convert --source postgres --target mysql ./input.sql
```

Single file with explicit input and output paths:

```
madsql convert --source postgres --target oracle --in ./input.sql --out ./converted
```

Directory tree with preserved relative layout:

```
madsql convert --source postgres --target mysql --in ./sql --out ./converted
```

Glob input:

```
madsql convert --source tsql --target postgres "./sql/**/*.sql" --out ./converted
```

If more than one input file is resolved, `--out` is required.

Output Naming

Converted files use deterministic names. For a normal conversion run, the output file is based on the input stem and target dialect:

```
input.postgres.sql
input.mysql.sql
```

You can override the suffix:

```
madsql convert \
  --source postgres \
  --target mysql \
  --in ./input.sql \
  --out ./converted \
  --suffix .converted.sql
```

Formatting

Compact SQL is the default.

Pretty-print converted SQL:

```
madsql convert --source postgres --target mysql ./input.sql --pretty
```

Be explicit about compact output:

```
madsql convert --source postgres --target mysql ./input.sql --compact
```

Split While Converting

If the output should be translated and also separated into one file per statement, use `--split-statements`:

```
madsql convert \
  --source postgres \
  --target oracle \
  --in ./input.sql \
  --out ./converted \
  --split-statements
```

This writes deterministic statement files such as:

```
0001_stmt.oracle.sql
0002_stmt.oracle.sql
```

Infer Schema While Converting

If you want translated SQL plus a schema side artifact in the same run:

```
madsql convert \
  --source postgres \
  --target mysql \
  --in ./sql \
  --out ./converted \
  --infer-schema
```

This writes a deterministic artifact at the base of `--out`, for example:

```
inferred_schema-postgres-to-mysql.sql
```

The infer-schema artifact options mirror the standalone schema command, but use `--infer-schema-*` names in the `convert` workflow.

Use hybrid inference for the side artifact when the workload includes statements that benefit from supplemental metadata or DDL parsing:

```
madsql convert \
  --source postgres \
```

```
  --target mysql \
  --in ./sql \
  --out ./converted \
  --infer-schema \
  --infer-schema-engine hybrid
```

This uses the same hybrid inference strategy as standalone `infer-schema` and requires `pip install -e .[infer]`.

Safe Batch Execution

Useful batch flags:

- `--overwrite` to allow replacing existing outputs
- `--errors errors.json` to capture failures in machine-readable form
- `--log 0` or `--log 1` to capture run metadata
- `--report` to write a Markdown summary

Example:

```
madsql convert \
  --source postgres \
  --target mysql \
  --in ./sql \
  --out ./converted \
  --continue \
  --log 1 \
  --report \
  --errors errors.json
```

Chapter 4: Infer Schema

`infer-schema` is the standalone schema extraction workflow. Use it when the main output should be recovered DDL or schema JSON, not converted SQL.

This is especially useful when:

- You inherited a workload without complete source DDL.
- You want a starter schema before a migration project begins.
- You need a best-effort model of referenced tables and columns from query traffic, reports, or hand-curated SQL collections.

Supported Statement Classes

`infer-schema` recognizes these statement classes:

- `USE`
- `SELECT`
- `DELETE`

- INSERT
- UPDATE
- CREATE SCHEMA
- CREATE DATABASE
- CREATE TABLE
- CREATE VIEW
- CREATE MATERIALIZED VIEW
- CREATE INDEX

Other parsed statement types are reported as unsupported and, by default, processing continues.

Basic Usage

Infer DDL from standard input:

```
cat queries.sql | madsql infer-schema --source singlestore
```

Infer DDL from one file:

```
madsql infer-schema --source singlestore ./examples/input/singlestore/nyc_taxi_queries.sql
```

Merge many files into one schema artifact:

```
madsql infer-schema --source postgres --in ./sql --out ./artifacts
```

Use hybrid inference when you want SQLGlot plus supplemental metadata and DDL recovery:

```
madsql infer-schema --source postgres --infer-engine hybrid --in ./sql --out ./artifacts
```

Output Modes

DDL is the default:

```
madsql infer-schema --source postgres ./queries.sql
```

JSON output:

```
madsql infer-schema --source postgres --format json --in ./sql --out ./artifacts
```

Pretty DDL:

```
madsql infer-schema --source postgres ./queries.sql --pretty
```

Explicit compact DDL:

```
madsql infer-schema --source postgres ./queries.sql --compact
```

When `--out` points to a directory, the output file uses a deterministic name such as:

```
inferred_schema-postgres.sql
inferred_schema-postgres.json
inferred_schema-postgres-to-mysql.sql
```

Inference Engines

SQLGlot-only inference is the default:

```
madsql infer-schema --source postgres ./queries.sql
```

Hybrid inference keeps SQLGlot as the primary parser and supplements it with `sql-metadata` for query metadata and `simple-ddl-parser` for DDL extraction:

```
madsql infer-schema --source postgres --infer-engine hybrid ./queries.sql
```

Use hybrid mode when the workload mixes normal SQLGlot-friendly SQL with DDL or metadata-only statements that still contain useful table and column signals. Hybrid mode requires `pip install -e .[infer]`.

Tuning Inference

Choose a fallback type when stronger evidence is unavailable:

```
madsql infer-schema --source postgres --default-type VARCHAR(255) ./queries.sql
```

Handle unqualified columns conservatively:

```
madsql infer-schema --source postgres --unqualified-columns skip ./queries.sql
```

Emit `CREATE TABLE IF NOT EXISTS`:

```
madsql infer-schema --source postgres --if-not-exists ./queries.sql
```

Common `--default-type` values include:

- `TEXT`
- `VARCHAR(255)`
- `VARCHAR2(100)`
- `NUMBER`
- `NUMBER(12)`
- `NUMBER(10,2)`
- `DOUBLE`
- `BIGINT`
- `DATE`
- `TIMESTAMP`
- `CLOB`
- `BLOB`
- `GEOGRAPHY`

Namespace and User DDL

For non-Oracle targets, prepend inferred schema creation statements:

```
madsql infer-schema \
  --source postgres \
  --target postgres \
  --create-schema \
  ./queries.sql
```

For Oracle targets, prepend user creation and grants:

```
madsql infer-schema \
  --source singlestore \
  --target oracle \
  --create-user \
  --create-user-password ChangeMe123 \
  ./queries.sql
```

Important rules:

- `--create-schema` is for non-Oracle DDL output.
- `--create-user` requires `--target oracle`.
- Oracle user creation requires `--create-user-password`.
- `--create-schema` and `--create-user` are DDL-only features, not JSON features.

Heuristic Behavior

`infer-schema` is intentionally best-effort when it works from query-only inputs.

What that means in practice:

- Explicit `CREATE TABLE` statements provide the strongest type evidence.
- `INSERT INTO table(col1, col2, ...)` helps establish table and column membership.
- `SELECT`, `UPDATE`, and view definitions contribute referenced tables and columns.
- Unqualified columns in multi-table queries can be assigned to the first table in scope or skipped, depending on `--unqualified-columns`.
- Low-confidence assignments are surfaced in DDL comments and JSON output.

Chapter 5: Reporting and Run Artifacts

Migration work is easier to trust when the run produces stable artifacts. `madsql` supports three kinds of reporting output:

- Structured error JSON via `--errors`
- Timestamped logs via `--log`

- Timestamped Markdown summaries via `--report`

JSON Error Reports

Use `--errors` when another program, a CI job, or a review workflow needs to consume failure information.

Example:

```
madsql convert \
  --source postgres \
  --target mysql \
  --in ./sql \
  --out ./converted \
  --errors errors.json
```

The JSON payload includes:

- `version_info`
- `errors`

Each error entry can include:

- `path`
- `statement_index`
- `error_type`
- `message`
- `statement_type`
- `statement_sql`

When `--out` is set and the `--errors` path is relative, the report is written to the base of `--out`. For example:

- Directory output: `./converted/errors.json`
- File output: alongside the explicit output file

Logs

Use `--log 0` for a compact machine-friendly summary and `--log 1` for more detail.

Typical log names:

```
20260312-180536-madsql-convert.log
20260312-191651-madsql-infer-schema.log
```

Logs include:

- Timestamp
- Full command line
- Command type

- Dialect and format settings
- Version information
- Success and failure counts
- Extra error detail at verbosity level `1`

`--log` requires `--out`.

Markdown Reports

Use `--report` when you want a human-readable run summary that can be attached to a ticket, a CI artifact bundle, or a review handoff.

Typical report names:

```
20260312-180536-madsql-convert-report.md
20260312-191651-madsql-infer-schema-report.md
20260312-191651-madsql-infer-schema-hybrid.md
```

Standard infer-schema runs keep the `YYYYMMDD-HHMMSS-madsql-infer-schema-report.md` name. Hybrid infer-schema runs use `YYYYMMDD-HHMMSS-madsql-infer-schema-hybrid.md`.

Reports summarize the run with metrics such as:

- Inputs processed
- Output files written
- Statement counts
- Success and failure totals
- Version information
- Type or confidence summaries for schema inference

In higher-detail scenarios, reports also capture failure details and SQL text when available.

`--report` requires `--out`.

Recommended Artifact Strategy

For interactive experimentation:

- Start with stderr and standard output only.

For repeatable migration runs:

- Add `--errors errors.json`
- Add `--log 1`
- Add `--report`

For CI or scheduled jobs:

- Prefer all three so machines can read the JSON and humans can read the Markdown summary.

Chapter 6: Debugging and Error Handling

`madsql` is designed to keep large runs inspectable rather than forcing you to start over after every failure.

Default Failure Behavior

By default:

- `convert` and `split-statements` continue through recorded conversion or split failures and report them at completion.
- `infer-schema` continues through unsupported and parse errors and keeps processing later statements.
- Fatal CLI misuse stops immediately.

Run Control Flags

Use `--continue` when you want default continuation behavior to be explicit in automation:

```
madsql convert --source postgres --target mysql --in ./sql --out ./converted --continue
```

Use `--fail-fast` when the first error should stop the run:

```
madsql convert --source postgres --target mysql --in ./sql --out ./converted --fail-fast
```

Use `--ignore-errors` when diagnostics should still be recorded but the final exit code should be `0`:

```
madsql infer-schema \
  --source postgres \
  --in ./sql \
  --out ./artifacts/schema.sql \
  --continue \
  --ignore-errors \
  --errors infer-errors.json
```

`--continue` and `--fail-fast` are mutually exclusive.

Exit Codes

`madsql` uses three important exit-code classes:

- `0`: success
- `1`: the command completed, but parse, conversion, or read errors were recorded
- `2`: CLI misuse, invalid argument combinations, or other fatal invocation problems

Common Problems and Fixes

Unsupported dialect name:

- Symptom: fatal CLI error before processing begins
- Fix: run `madsql dialects` and use an exact supported name

Missing `--out`:

- Symptom: fatal CLI error for multi-input runs, split output, `--report`, `--log`, or schema side artifacts
- Fix: provide an explicit output file or directory

Overwrite refusal:

- Symptom: output file already exists
- Fix: choose a new output path or add `--overwrite`

Ambiguous query-only inference:

- Symptom: low-confidence columns or generic fallback types
- Fix: prefer inputs with `CREATE TABLE`, tune `--default-type`, or set `--unqualified-columns skip`

Oracle schema-generation misuse:

- Symptom: fatal error when combining Oracle output with the wrong namespace flag
- Fix: use `--create-user --create-user-password ...` for Oracle; use `--create-schema` only for non-Oracle targets

Single problematic script in a large batch:

- Symptom: one file keeps failing inside a directory run
- Fix: rerun the file by itself, then use `split-statements` to isolate the statement index causing the failure

Practical Debugging Workflow

When a run fails, use this sequence:

1. Start with one file instead of a directory or glob.
2. Add `--errors errors.json` so the failure becomes machine-readable.
3. Add `--log 1 --report` when the run is large enough that stderr alone is not sufficient.
4. If conversion is failing inside a large script, run `split-statements` on that script and inspect the statement indexes.
5. If schema inference looks wrong, look for low-confidence comments and then adjust `--default-type` or `--unqualified-columns`.
6. Once the behavior is understood, scale back out to the directory or glob run.

A Real Parse Failure Example

This command returns exit code `1` and writes a JSON error report:

```
madsql convert \
  --source postgres \
  --target mysql \
  --in ./bad.sql \
  --out ./converted \
  --errors errors.json
```

A parse failure is surfaced with the input path, statement index, error type, and parser message. On stderr you will also see a version summary such as:

```
debug_versions: madsql=0.11.0, python=3.13.5, sqlglot=29.0.1, sqlparse=0.5.3
```

That makes it easier to compare runs across machines and CI environments.

Final Notes

Start small, verify the translated or inferred output on representative SQL, and then scale up to directory or glob workflows once the flags match your project's expectations.

Recommended first commands:

```
madsql dialects
madsql convert --help
madsql split-statements --help
madsql infer-schema --help
```